

Atividade Formativa: ANÁLISE DE REQUISITOS E METODOLOGIAS ÁGEIS (SCRUM/KANBAN)

Nome: Gabriel Lúcio de Araújo

Data: 25/03

Instruções: Responda às questões dissertativas de forma clara, utilizando os conceitos de Scrum, Kanban e Engenharia de Requisitos discutidos em aula.

PARTE 1: DINÂMICA SCRUM E KANBAN

- 1. O Papel do PO no Scrum:** Como o Product Owner deve atuar na transformação de uma necessidade de negócio (ex: "quero rastrear frotas") em um item pronto para o desenvolvimento no Backlog?

Resumo da atuação do PO:

Entender: "Por que rastrear?"

Narrar: Escrever a história (Quem, O quê, Por quê).

Refinar: Criar critérios de aceite (Como validar).

Priorizar: Colocar no topo se for alto valor.

Colaborar: Tirar dúvidas com o time

O **Scrum** é um método ágil focado no **gerenciamento do desenvolvimento iterativo** de software. Ele organiza o projeto em três fases principais: **planejamento geral** (metas e arquitetura), **ciclos de sprint** (onde o software é desenvolvido) e **encerramento** (documentação e lições aprendidas)

O Product Owner traduz a necessidade de negócio em valor claro para o cliente, definindo o objetivo do produto.

Refina a ideia em itens menores (Product Backlog Items), com descrição, critérios de aceitação e prioridade.

Colabora com stakeholders e time para esclarecer dúvidas e alinhar expectativas.

Garante que os itens estejam compreensíveis, testáveis e prontos para desenvolvimento.

Prioriza continuamente o backlog com base em valor, risco e feedback.

- 2. User Stories vs. Tarefas:** Explique a diferença entre uma *História de Usuário* (focada no valor) e as *Tarefas Técnicas* (focada no "como fazer") dentro do contexto de uma Sprint Planning.

História de Usuário (User Story):

Representa o que deve ser entregue e por quê, focando no valor para o usuário/negócio.

Geralmente segue o formato: "Como [usuário], quero [funcionalidade] para [benefício]".

É responsabilidade do Product Owner e guia o objetivo da Sprint.

Tarefas Técnicas:

São a decomposição da história em atividades práticas de implementação (como fazer).

Criadas e detalhadas pelo time de desenvolvimento durante o Sprint Planning.

Incluem ações como desenvolver, testar, integrar, etc.

Diferença no contexto da Sprint Planning:

A História define o valor a ser entregue, enquanto as Tarefas definem como o time vai entregar.
A história orienta; as tarefas operacionalizam.

Resposta resumida (para estudo rápido)

História de Usuário foca no valor e no o que/por quê será entregue.
Tarefas Técnicas focam no como fazer a implementação.
A história guia o objetivo da Sprint.
As tarefas detalham o trabalho necessário para realizá-la.
Juntas, conectam valor de negócio à execução técnica.

3. **Critérios de Aceite:** Por que definir "Critérios de Aceite" claros é fundamental para evitar que um requisito seja reaberto após o fim da Sprint?

Critérios de Aceite:

São condições claras e objetivas que definem quando uma História de Usuário está concluída.
Descrevem como validar a funcionalidade do ponto de vista do negócio/usuário.
Servem como base para testes e validação pelo time e stakeholders.

Importância dentro da Sprint:

Alinham expectativas entre Product Owner, time e stakeholders antes do desenvolvimento.
Evitam interpretações diferentes sobre o que significa "pronto".
Guiam o desenvolvimento e os testes durante a Sprint.

Relação com retrabalho:

Sem critérios claros, o time pode entregar algo diferente do esperado.
Isso gera ajustes após a Sprint, reabrindo o item e causando retrabalho.

Resposta resumida (para estudo rápido)

Critérios de Aceite definem claramente quando uma entrega está pronta.
Alinham expectativas entre PO, time e stakeholders.
Reduzem ambiguidades e interpretações erradas.
Guiam testes e validação da funcionalidade.
Evitam retrabalho e reabertura de requisitos após a Sprint.

4. **O "Definition of Ready" (DoR):** Como a falta de uma definição de "Requisito Pronto" (DoR) pode impactar negativamente a velocidade do time durante a Sprint?

Definition of Ready (DoR):

É um conjunto de critérios que define quando um item do backlog está pronto para ser desenvolvido.
Inclui clareza de escopo, critérios de aceite definidos, dependências identificadas e tamanho adequado.
Garante que o time comece a Sprint com itens bem entendidos.

Impacto na Sprint:

Sem DoR, itens entram na Sprint incompletos ou confusos.
O time perde tempo tirando dúvidas, refinando ou redefinindo o trabalho durante a execução.
Isso gera interrupções, bloqueios e mudanças de escopo.

Impacto na velocidade:

Aumenta retrabalho, reduz previsibilidade e dificulta cumprir o que foi planejado.

A velocidade (quantidade de trabalho entregue) cai devido à ineficiência e desperdício de tempo.

Resposta resumida (para estudo rápido)

Sem DoR, itens entram na Sprint sem clareza suficiente.

O time perde tempo refinando durante a execução.

Aumentam dúvidas, bloqueios e retrabalho.

Diminui a previsibilidade da entrega.

Isso reduz a velocidade e eficiência do time.

5. **Priorização no Backlog:** No Scrum, o Backlog é ordenado por valor. Se o cliente pedir dois requisitos conflitantes, quais critérios o PO deve usar para decidir qual vem primeiro?

Priorização por valor:

No Scrum, o Product Owner ordena o backlog com base no valor gerado ao negócio/usuário.

Valor pode envolver receita, satisfação do cliente, redução de custos ou aprendizado.

Requisitos conflitantes:

Ocorrem quando duas demandas competem por recursos, tempo ou têm objetivos opostos.

Exigem decisão estratégica do PO, considerando impacto e contexto do produto.

Critérios de decisão:

Valor de negócio: qual gera mais impacto positivo imediato ou estratégico.

Urgência: prazos, demandas legais ou oportunidades de mercado.

Risco: reduzir incertezas ou evitar problemas futuros.

Dependências: um item pode destravar outros.

Custo/Esforço vs. benefício: melhor relação entre ganho e esforço.

Feedback do cliente/mercado: o que traz mais aprendizado ou validação.

Resposta resumida (para estudo rápido)

O PO prioriza pelo maior valor de negócio.

Considera urgência, risco e dependências.

Avalia custo versus benefício de cada item.

Leva em conta feedback do cliente e estratégia.

Escolhe o item que gera mais impacto com menor risco.

6. **Visualização com Kanban:** Como o uso de um quadro Kanban (com colunas como To Do, Doing, Testing, Done) ajuda a identificar se o problema do projeto está na definição dos requisitos ou na execução técnica?

Quadro Kanban:

Ferramenta visual que mostra o fluxo de trabalho em colunas (To Do, Doing, Testing, Done).

Permite acompanhar o progresso das tarefas em tempo real e identificar gargalos.

Fluxo de trabalho:

Representa o caminho que um item percorre até ser concluído.

Ajuda a entender onde o trabalho está acumulando ou travando.

Identificação de problemas:

Se muitos itens ficam em To Do: pode indicar requisitos mal definidos ou falta de clareza (problema no refinamento/PO).

Se acumulam em Doing: pode indicar dificuldades técnicas ou excesso de trabalho em progresso.

Se travam em Testing: pode indicar problemas de qualidade ou falhas nos critérios de aceite.

Resposta resumida (para estudo rápido)

O Kanban mostra onde o trabalho está travando.

Acúmulo em To Do indica problema na definição dos requisitos.

Acúmulo em Doing/Testing indica problema na execução técnica.

Ajuda a visualizar gargalos no fluxo.

Facilita decisões para melhorar o processo.

7. **WIP Limit (Limite de Trabalho em Foco):** No Kanban, limitamos o trabalho em andamento. Como essa prática ajuda a equipe a terminar um requisito antes de começar o próximo?

WIP Limit (Work In Progress):

É o limite de quantidade de tarefas que podem estar em andamento ao mesmo tempo.

Evita que o time comece muitas coisas sem terminar.

Foco e fluxo contínuo:

Com menos itens em progresso, o time consegue se concentrar melhor.

Reduz multitarefa e troca constante de contexto.

Impacto na entrega:

O time passa a priorizar finalizar tarefas antes de puxar novas.

Isso diminui gargalos e acelera a entrega de valor.

Resposta resumida (para estudo rápido)

WIP limita quantas tarefas ficam em andamento.

Reduz multitarefa e perda de foco.

Força o time a terminar antes de começar novas tarefas.

Diminui gargalos no fluxo de trabalho.

Aumenta a eficiência e a velocidade de entrega.

8. **Mudanças de Escopo:** Se um requisito novo e urgente surgir durante a Sprint do Scrum, qual o procedimento padrão? E como o Kanban lidaria com essa mesma urgência de forma diferente?

Mudanças de escopo no Scrum:

Durante a Sprint, o escopo deve ser estável para garantir foco e previsibilidade.

Mudanças só são aceitas se não comprometerem o objetivo da Sprint.

O Product Owner negocia com o time: pode substituir itens ou, em casos extremos, cancelar a Sprint.

Procedimento padrão no Scrum:

O novo requisito é avaliado e normalmente vai para o backlog para a próxima Sprint. Se for muito urgente, o PO e o time ajustam o escopo atual ou interrompem a Sprint.

Kanban e urgência:

Kanban é um fluxo contínuo, sem Sprints fixas.

Permite inserir itens urgentes diretamente no fluxo, respeitando limites de WIP.

Pode usar classes de serviço (ex: “expedite”) para priorizar urgências.

Diferença principal:

Scrum protege a Sprint; Kanban é mais flexível e adaptável a mudanças imediatas.

Resposta resumida (para estudo rápido)

No Scrum, mudanças entram no backlog para próxima Sprint.

Se urgente, o PO negocia ajuste ou pode cancelar a Sprint.

Scrum prioriza estabilidade da Sprint.

No Kanban, a mudança entra direto no fluxo.

Kanban é mais flexível para lidar com urgências.

9. **Gargalos no Fluxo:** Imagine que a coluna "Testes" do seu Kanban está lotada, mas a coluna "Desenvolvimento" está vazia. O que isso indica sobre a qualidade dos requisitos entregues?

Gargalos no fluxo (Kanban):

Um gargalo ocorre quando uma etapa do processo acumula mais trabalho do que consegue processar.

No quadro Kanban, isso aparece como acúmulo de itens em uma coluna específica, atrasando a entrega.

Interpretação do cenário (Testes lotada e Desenvolvimento vazio):

Se a coluna de Testes está cheia e Desenvolvimento está vazia, significa que o time está gerando mais trabalho em validação do que consegue finalizar.

Isso pode indicar que os itens entregues em desenvolvimento não estão prontos para teste (problemas de qualidade).

Qualidade dos requisitos entregues:

Pode haver problemas como:

Falta de critérios de aceite claros

Bugs ou falhas frequentes

Baixa qualidade técnica ou incompletude das entregas

Itens que retornam para retrabalho

Resposta resumida (para estudo rápido)

Testes lotada e Desenvolvimento vazio indica um gargalo na validação.

Sugere que o trabalho não está fluindo bem após o desenvolvimento.

Pode apontar baixa qualidade nas entregas.

Há possível retrabalho por falhas ou critérios de aceite mal definidos.

Isso impacta a eficiência e a velocidade do fluxo.

10. **Sprint Review e Feedback:** Explique como a reunião de Review serve como uma etapa de "re-análise de requisitos" baseada no que o cliente viu no incremento do software.

Sprint Review:

É a cerimônia do Scrum onde o time apresenta o incremento do produto ao Product Owner e stakeholders.

O objetivo é inspecionar o que foi entregue e coletar feedback real sobre o produto.

Reanálise de requisitos:

Após ver o incremento, os stakeholders podem validar, sugerir mudanças ou ajustar expectativas.

Isso pode gerar novos insights, refinamento de requisitos existentes ou até novas demandas.

O Product Owner atualiza e reprioriza o Product Backlog com base nesse feedback.

Relação entre Review e requisitos:

A Review funciona como um ponto de inspeção prática dos requisitos.

Em vez de discutir apenas teoria, os requisitos são avaliados na forma de software funcionando.

Isso reduz ambiguidades e alinha melhor o produto às necessidades reais do cliente.

Resposta resumida (para estudo rápido)

A Sprint Review permite ao cliente ver o incremento funcionando.

Com isso, ele valida ou ajusta os requisitos na prática.

O feedback gera novas percepções e possíveis mudanças.

O PO atualiza e reprioriza o backlog com base nisso.

Funciona como uma reanálise contínua dos requisitos.

PARTE 2: ENGENHARIA E ANÁLISE DE REQUISITOS

11. **Elicitação de Requisitos:** Imagine que os stakeholders não sabem explicar o que precisam. Quais técnicas de elicitación você utilizaria para extrair requisitos reais para o Backlog?

Elicitação de Requisitos:

É o processo de descobrir, levantar e entender as necessidades reais dos stakeholders.

Quando eles não conseguem expressar claramente o que precisam, o PO usa técnicas para extrair informações implícitas.

Técnicas de elicitación:

Entrevistas: conversas estruturadas para explorar necessidades e problemas.

Workshops/JAD: reuniões colaborativas com stakeholders para alinhar expectativas e ideias.

Observação (shadowing): acompanhar o usuário no dia a dia para entender seu contexto real.

Prototipação: criar protótipos ou wireframes para validar ideias rapidamente.

Brainstorming: gerar ideias coletivas e explorar possibilidades.

Análise de documentos/dados: uso de relatórios, sistemas existentes e métricas.

Objetivo:

Transformar necessidades vagas em requisitos claros, priorizáveis e testáveis no backlog.

Resposta resumida (para estudo rápido)

Usaria entrevistas para explorar necessidades dos stakeholders.

Workshops para alinhar ideias em grupo.

Observação para entender o uso real.

Prototipação para validar rapidamente as soluções.

Essas técnicas ajudam a transformar necessidades vagas em requisitos claros para o backlog.

12. Requisitos: Defina o que são requisitos funcionais e não funcionais:**Requisitos Funcionais:**

Descrevem o que o sistema deve fazer.

Representam funcionalidades, comportamentos e regras de negócio.

Exemplos: login de usuário, cálculo de frete, cadastro de produtos, geração de relatórios.

Estão diretamente ligados às interações do usuário com o sistema.

Requisitos Não Funcionais:

Descrevem como o sistema deve se comportar ou suas qualidades.

Não definem funcionalidades, mas atributos de qualidade do sistema.

Exemplos: desempenho (tempo de resposta), segurança, escalabilidade, usabilidade, disponibilidade.

Impactam a experiência do usuário e a arquitetura do sistema.

Diferença principal:

Funcionais = funcionalidades do sistema (o que faz).

Não funcionais = qualidade e restrições (como faz).

Resposta resumida (para estudo rápido)

Requisitos funcionais definem o que o sistema faz.

Ex: login, cadastro, cálculo de dados.

Requisitos não funcionais definem como o sistema deve se comportar.

Ex: desempenho, segurança, usabilidade.

Funcionais = funcionalidades; não funcionais = qualidade do sistema.

13. Requisitos Funcionais vs. Não-Funcionais: Como você documentaria requisitos de desempenho (ex: tempo de resposta do sistema) dentro de uma User Story de forma que o time saiba como testá-los?**Requisitos de desempenho (não funcionais):**

Estão relacionados à velocidade, tempo de resposta, carga e escalabilidade do sistema.

Exemplo: “o sistema deve responder em até 2 segundos para 95% das requisições”.

Precisam ser mensuráveis e testáveis para evitar ambiguidades.

User Story com critérios de aceitação:

A User Story descreve o valor (o que e por quê), enquanto os critérios de aceitação detalham condições claras para validação.

É nos critérios de aceitação que requisitos não funcionais devem ser explicitados de forma objetiva.

Documentação para testes:

Requisitos de desempenho devem ser escritos como critérios mensuráveis, incluindo métricas, limites e condições de teste.

Isso permite que o time de QA valide com testes de performance (ex: carga, stress, tempo de resposta).

Exemplo:

“O sistema deve responder em até 2 segundos para 95% das requisições sob carga de 100 usuários simultâneos.”

“O tempo médio de resposta não deve ultrapassar 1 segundo.”

Resposta resumida (para estudo rápido)

Requisitos de desempenho devem ser incluídos como critérios de aceitação na User Story.

Devem ser mensuráveis, como tempo máximo de resposta e percentual de requisições.

Exemplo: resposta em até 2 segundos para 95% das chamadas.

Isso permite testes claros de performance pelo time.

Assim, o time sabe exatamente como validar o requisito.

- 14. Análise de Impacto e Dependências:** Por que é arriscado iniciar o desenvolvimento de um requisito no Kanban que depende de um serviço externo ainda não concluído? Como o PO deve gerenciar isso?

Dependências no Kanban:

Ocorrem quando uma tarefa depende da conclusão de outra (interna ou externa) para poder avançar.

Serviços externos ainda não concluídos representam um bloqueio potencial no fluxo de trabalho.

Riscos de iniciar sem dependência resolvida:

Bloqueios durante o desenvolvimento

Retrabalho ou refatoração caso o serviço mude

Atrasos na entrega por falta de integração

Aumento de trabalho em andamento (WIP) parado

Perda de eficiência e desperdício de esforço

Análise de impacto:

Consiste em avaliar como uma mudança ou requisito afeta outras partes do sistema.

Ajuda a identificar riscos, dependências e prioridades antes de iniciar o desenvolvimento.

Gestão pelo Product Owner:

Identificar e mapear dependências no backlog

Priorizar itens que destravam outros (reduzir riscos)

Trabalhar alinhamento com equipes externas

Reordenar o backlog considerando dependências

Evitar iniciar itens bloqueados ou tratá-los com estratégia (ex: mocks, contratos de API)

Resposta resumida (para estudo rápido)

Iniciar um requisito com dependência externa pode causar bloqueios e retrabalho.

O time pode ficar parado aguardando o serviço ficar pronto.

Isso aumenta atrasos e reduz eficiência.

O PO deve identificar dependências e priorizar itens que destravam o fluxo.

Também deve alinhar com equipes externas e evitar iniciar itens bloqueados.

- 15. Refinamento (Grooming):** Durante o refinamento, o time diz que um requisito está "muito vago". Qual é a responsabilidade do PO e do Time para que ele chegue à Sprint Planning com o detalhamento adequado?

Refinamento (Grooming):

É a prática contínua de revisar, detalhar e preparar itens do Product Backlog para futuras Sprints. O objetivo é garantir que os itens estejam claros, priorizados e com informações suficientes para serem desenvolvidos.

Requisito “vago”:

Indica que a User Story não está bem definida, podendo faltar contexto, critérios de aceitação, regras de negócio ou entendimento comum.

Isso aumenta o risco de retrabalho e interpretações diferentes durante a Sprint.

Responsabilidade do Product Owner (PO):

Esclarecer o objetivo do requisito e o valor de negócio

Detalhar critérios de aceitação

Alinhar expectativas com stakeholders

Priorizar e ajustar o backlog conforme o entendimento evolui

Responsabilidade do Time de Desenvolvimento:

Fazer perguntas e apontar ambiguidades

Sugerir quebras da história em itens menores, se necessário

Ajudar a identificar cenários técnicos e riscos

Validar se o item está compreensível e viável para desenvolvimento

Objetivo conjunto:

Chegar à Sprint Planning com itens claros, estimados e prontos, reduzindo incertezas.

Resposta resumida (para estudo rápido)

O PO deve esclarecer o objetivo, detalhar critérios de aceitação e alinhar o negócio.

O time deve fazer perguntas, apontar ambiguidades e sugerir melhorias.

Ambos trabalham juntos no refinamento.

O objetivo é tornar o requisito claro, compreensível e testável.

Assim, ele chega pronto para a Sprint Planning.

16. Validação com Protótipos: De que forma o uso de protótipos de baixa fidelidade ajuda a validar requisitos antes que o time comece a codificar, evitando desperdício no fluxo Kanban?

Protótipos de baixa fidelidade:

São representações simples da interface ou solução (ex: wireframes, sketches, mockups básicos).

Não possuem implementação completa, focando apenas na ideia e no fluxo.

Servem para simular a experiência do usuário de forma rápida e econômica.

Validação de requisitos:

Consiste em confirmar com stakeholders se o que foi entendido atende às necessidades reais.

O protótipo permite visualizar e testar o conceito antes do desenvolvimento.

Redução de desperdício no Kanban:

Ao validar antes de codificar, evita-se desenvolver funcionalidades incorretas ou mal interpretadas.

Isso reduz retrabalho, filas de correção e itens retornando no fluxo.

Melhora a qualidade dos itens que entram em desenvolvimento, mantendo o fluxo mais eficiente.

Resposta resumida (para estudo rápido)

Protótipos de baixa fidelidade permitem visualizar a solução antes do desenvolvimento.

Ajudam a validar requisitos com stakeholders de forma rápida.

Identificam erros ou ajustes antes da codificação.

Reduzem retrabalho e desperdício no fluxo Kanban.

Aumentam a qualidade dos itens que entram em desenvolvimento.

17. Regra de negócio: Defina o que são regras de negócios e como elas interferem no levantamento de requisitos.

Regras de negócio:

São diretrizes, restrições ou condições que definem como um sistema deve operar dentro do contexto da empresa.

Orientam decisões e comportamentos do sistema com base nas políticas do negócio.

Exemplos: limites de desconto, validação de cadastro, regras de cálculo de preço, permissões de acesso.

Interferência no levantamento de requisitos:

As regras de negócio ajudam a transformar necessidades abstratas em requisitos claros e implementáveis.

Elas influenciam diretamente as regras funcionais da User Story e os critérios de aceitação.

Também ajudam a evitar ambiguidades, garantindo que todos entendam as condições corretas de funcionamento.

Sem regras bem definidas, os requisitos podem ficar incompletos ou inconsistentes.

Resposta resumida (para estudo rápido)

Regras de negócio são diretrizes que definem como o sistema deve operar.

Ex: limites, validações e condições de cálculo.

Elas orientam a definição de requisitos funcionais.
Ajudam a detalhar critérios de aceitação.
Reduzem ambiguidades e garantem consistência nos requisitos.

18. Brainstorm: Qual o papel do brainstorm na elaboração de um projeto de software?

Brainstorm (tempestade de ideias):

É uma técnica colaborativa usada para gerar ideias de forma livre e rápida, sem julgamentos iniciais.

Envolve stakeholders, PO e time para explorar diferentes soluções, funcionalidades e abordagens.

Papel no projeto de software:

Ajuda a levantar ideias iniciais de requisitos, funcionalidades e melhorias.

Estimula a criatividade e amplia a visão sobre o problema a ser resolvido.

Permite identificar diferentes perspectivas e possíveis soluções antes de tomar decisões.

Impacto no levantamento de requisitos:

O brainstorm contribui para descobrir necessidades que não foram explicitamente mencionadas.

As ideias geradas são posteriormente refinadas, priorizadas e transformadas em itens de backlog.

Serve como ponto de partida, não como definição final dos requisitos.

Resposta resumida (para estudo rápido)

O brainstorm é usado para gerar ideias de forma colaborativa e sem julgamentos.

Ajuda a explorar funcionalidades e soluções no início do projeto.

Permite levantar necessidades que não estavam explícitas.

As ideias são depois refinadas e priorizadas no backlog.

Funciona como base para definição inicial dos requisitos.

19. Questionário e entrevista: Diferencie Questionário e Entrevista, métodos utilizados na engenharia de software.

Questionário:

É um conjunto de perguntas estruturadas, geralmente padronizadas, respondidas por várias pessoas.

Pode ser aplicado de forma remota (online) e permite coletar dados de forma rápida e em grande escala.

As perguntas são normalmente fechadas (múltipla escolha, escalas) ou com respostas curtas.

Vantagem: eficiência e alcance. Limitação: menor profundidade nas respostas.

Entrevista:

É uma conversa direta (presencial ou remota) entre o analista/PO e o stakeholder.

Permite explorar respostas em profundidade, com perguntas abertas e follow-ups.

Facilita o esclarecimento imediato de dúvidas e a descoberta de detalhes implícitos.

Vantagem: profundidade e interação. Limitação: maior custo de tempo e esforço.

Diferença principal:

Questionário coleta dados amplos e padronizados; entrevista coleta informações mais detalhadas e contextualizadas.

Resposta resumida (para estudo rápido)

Questionário usa perguntas padronizadas para muitas pessoas, com respostas rápidas e objetivas.

Entrevista é uma conversa direta que permite aprofundar respostas.

Questionário tem maior alcance; entrevista tem maior profundidade.

Questionário é mais rápido; entrevista é mais detalhada.

A escolha depende do nível de detalhe necessário.